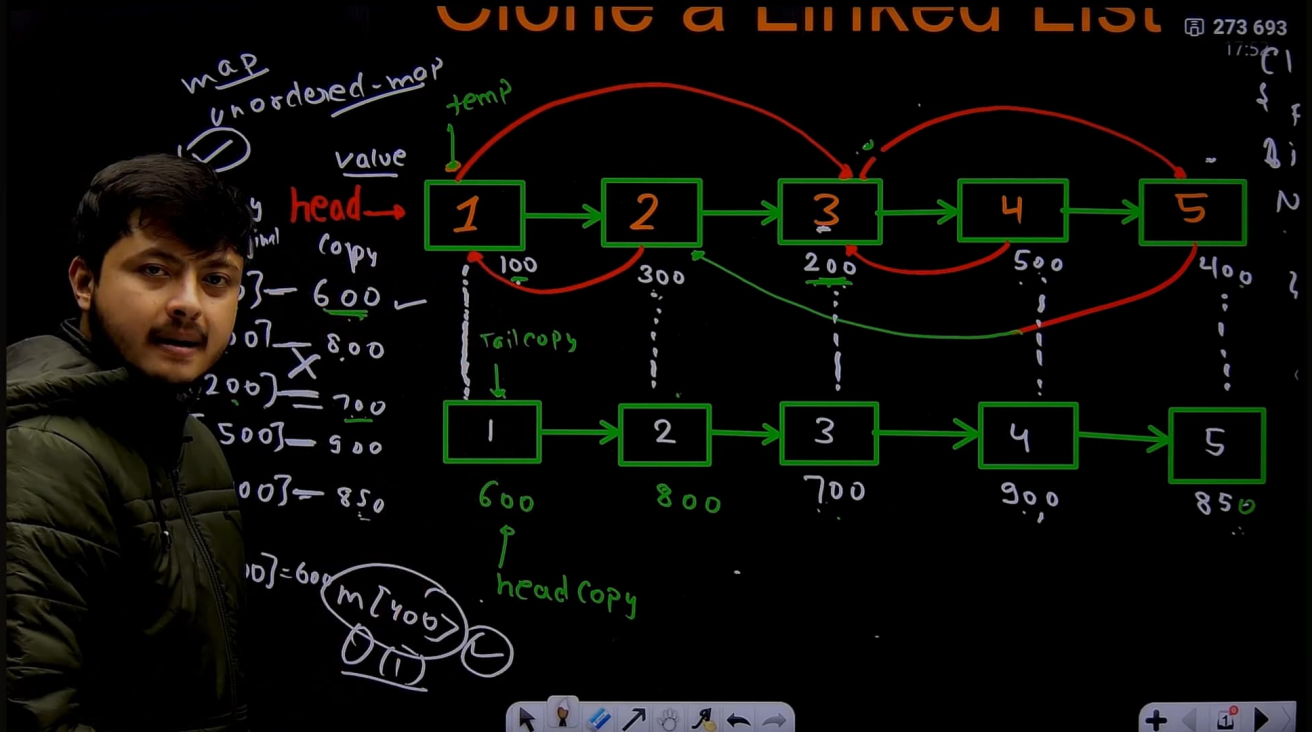
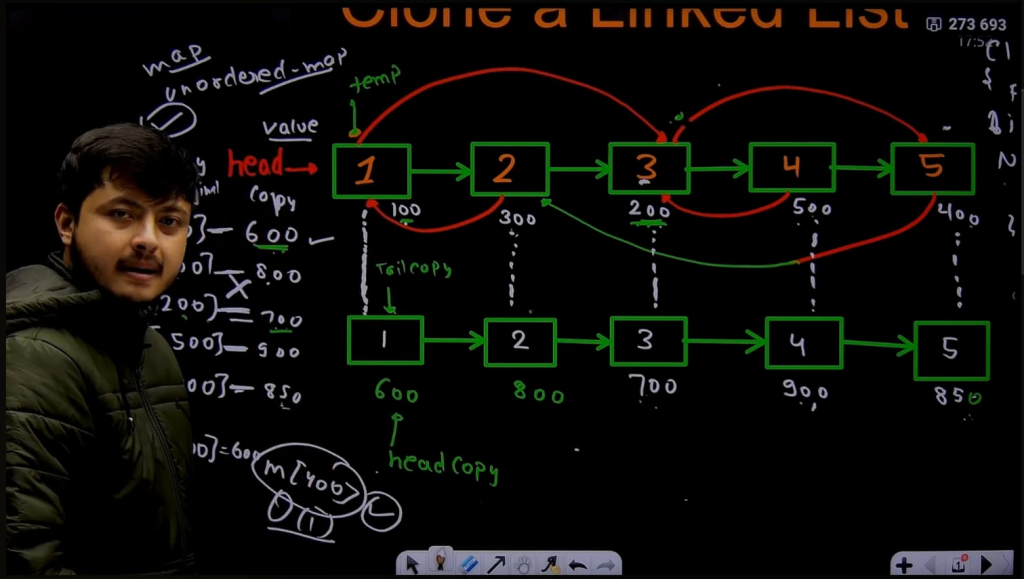
# Clone a Linked List

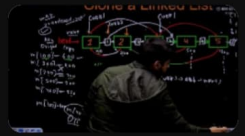




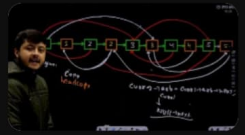

## In this video

Chapters | Transcript

Most Optimized Approach - Clone a Linked ... 48:05

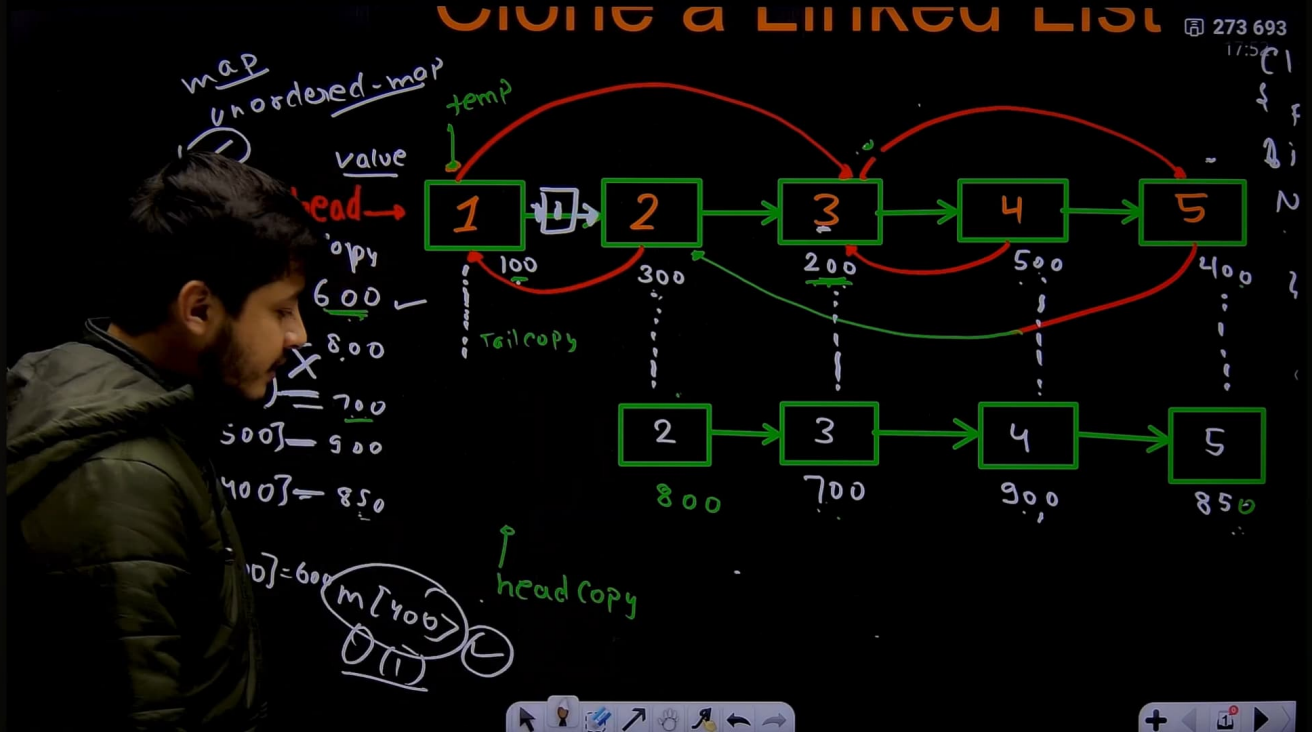


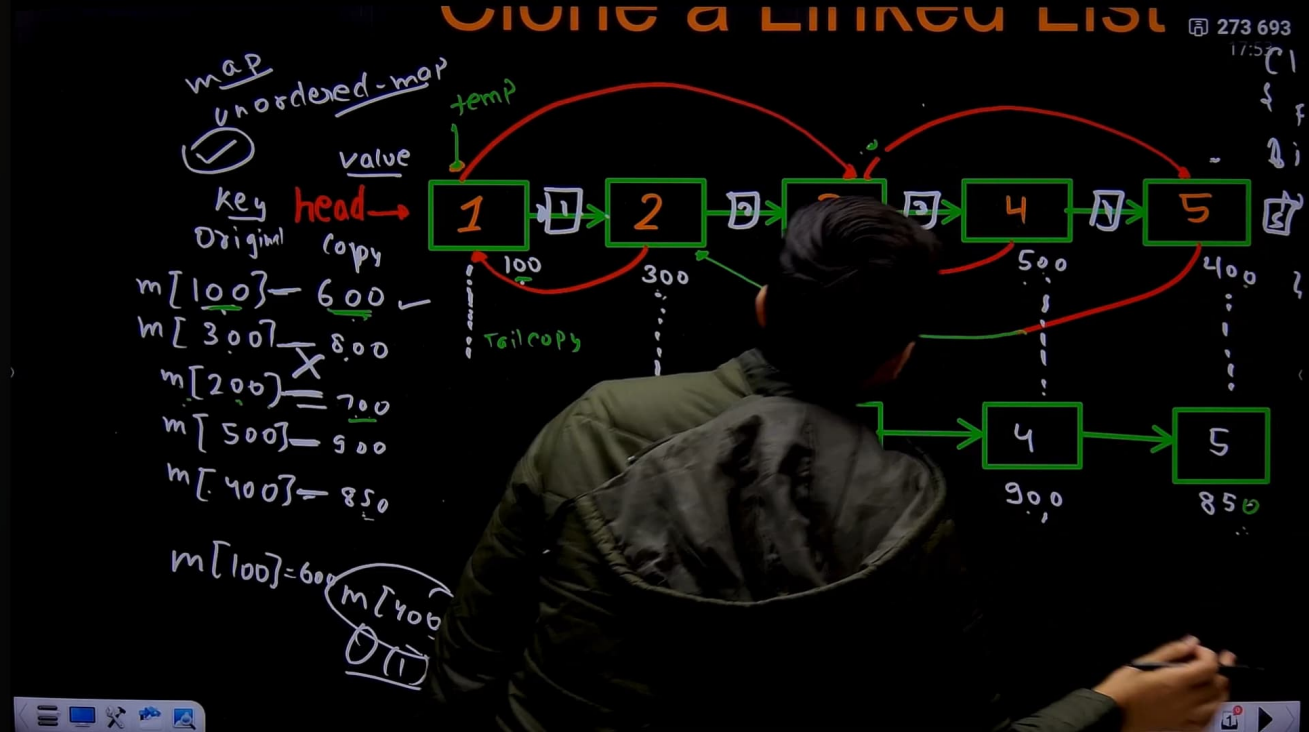
Discussing Edge Case 1:04:25

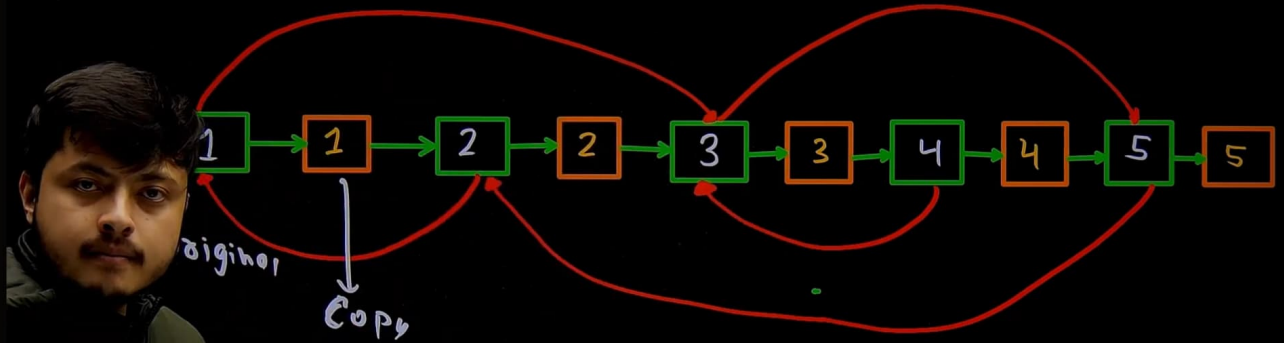


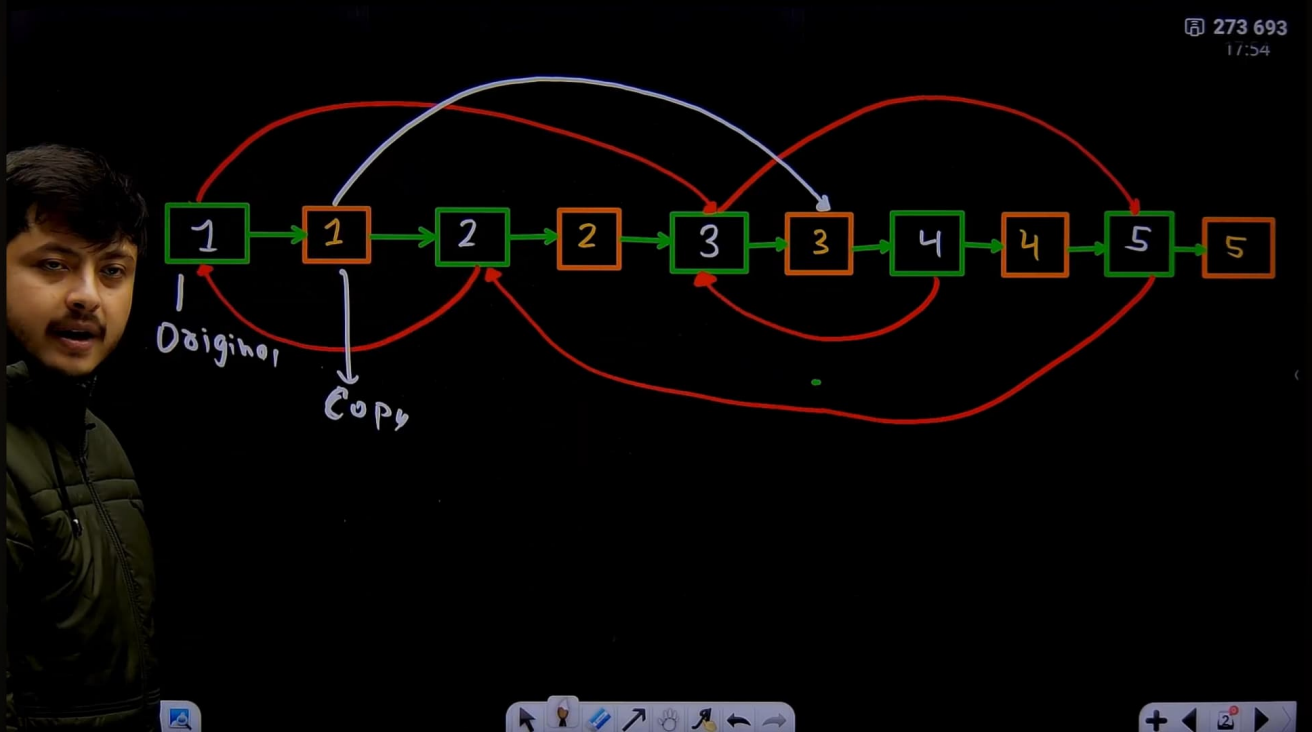
Continuation of Most Optimized Approach - Clone a Linked List 1:05:30

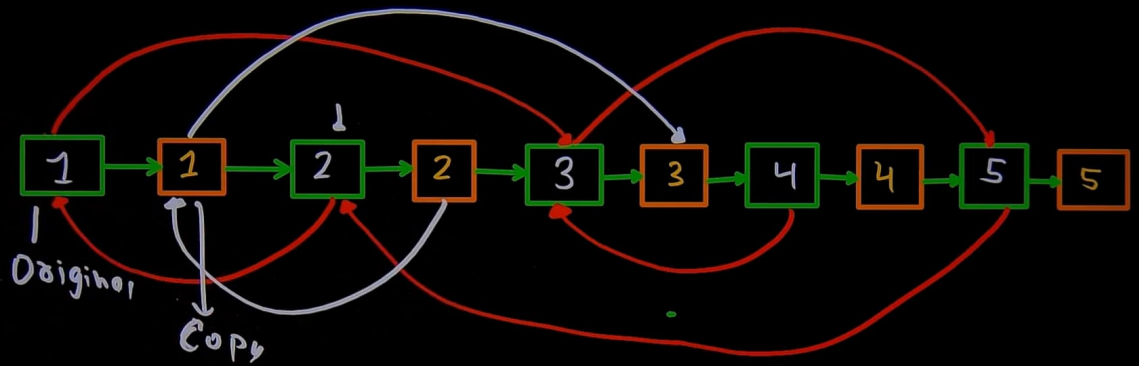
# Clone a Linked List

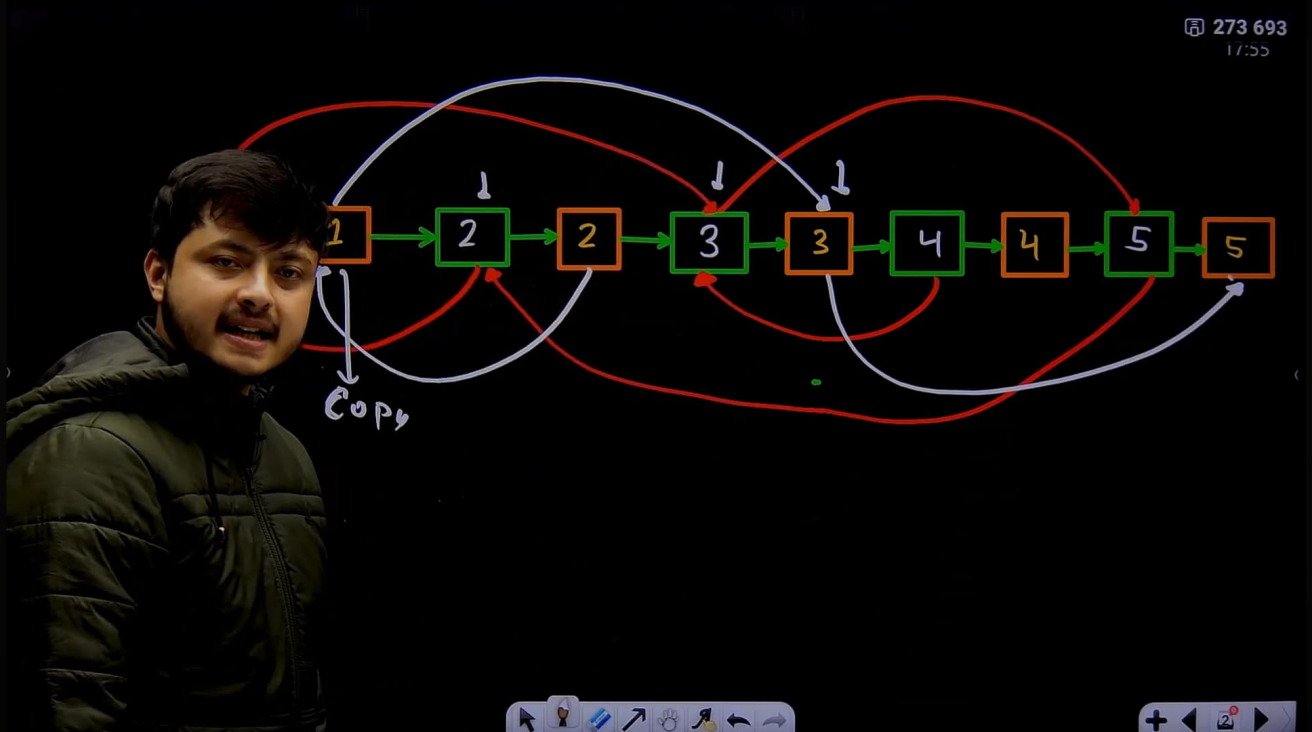


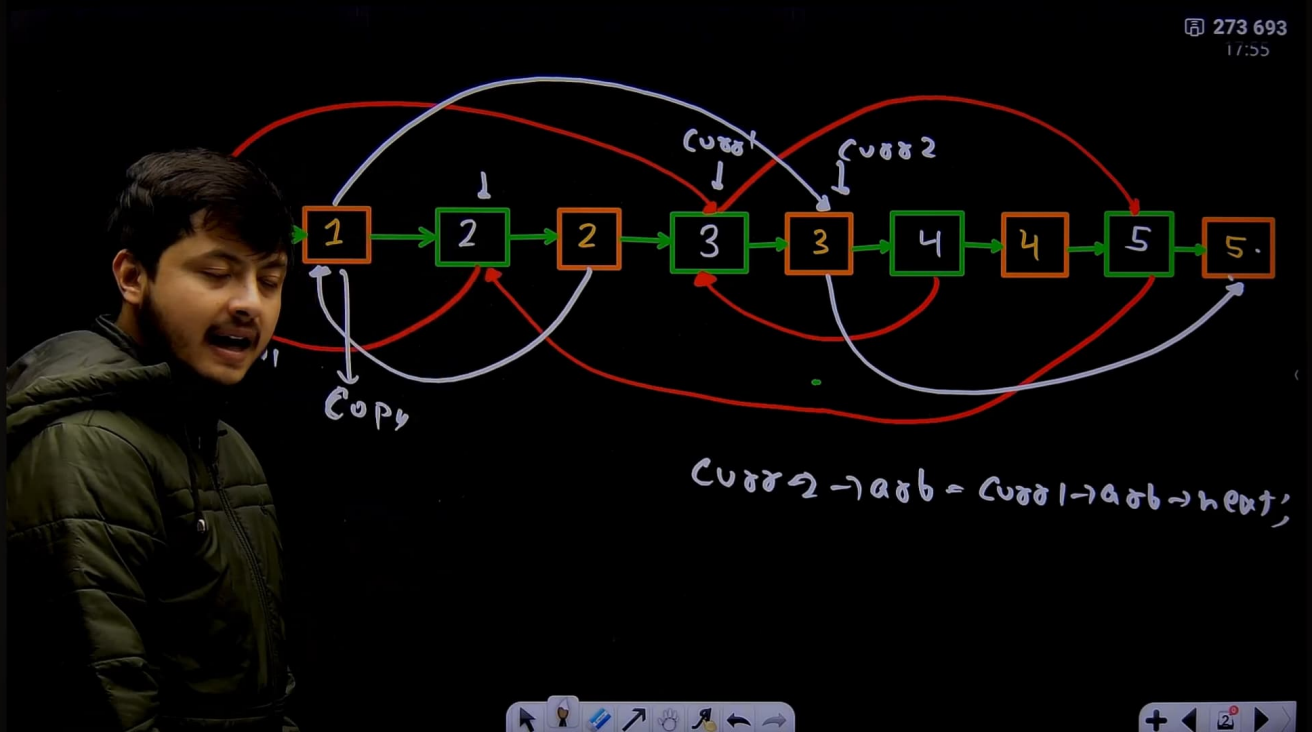

Clone a Linked List
273 693
map
unordered-map
temp
value
key
Original
head
copy
1
2
4
5
100
300
500
400
Tail copy
m[100] — 600
m[300] — 800
m[200] — 700
m[500] — 900
m[400] — 850
m[100]=600
m[400]
4
5
900
850

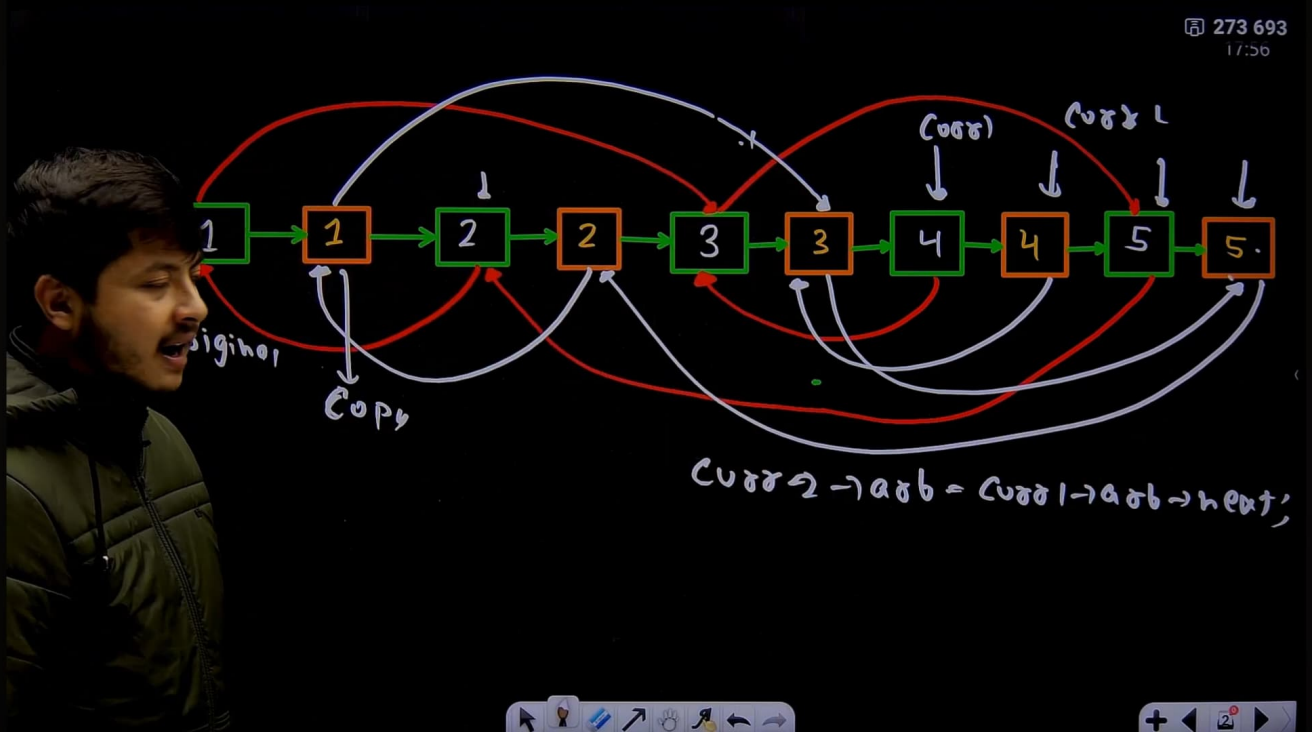


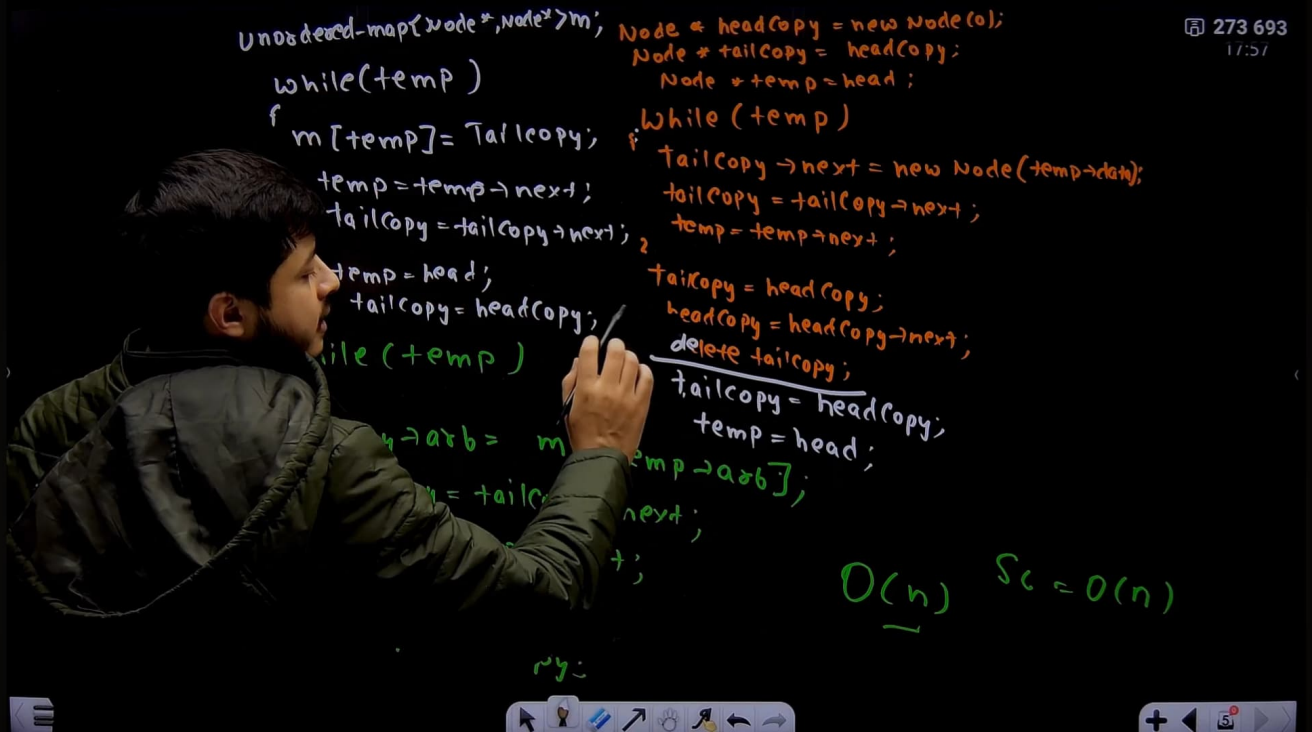

1
1
2
2
3
3
4
4
5
5
Original
Copy

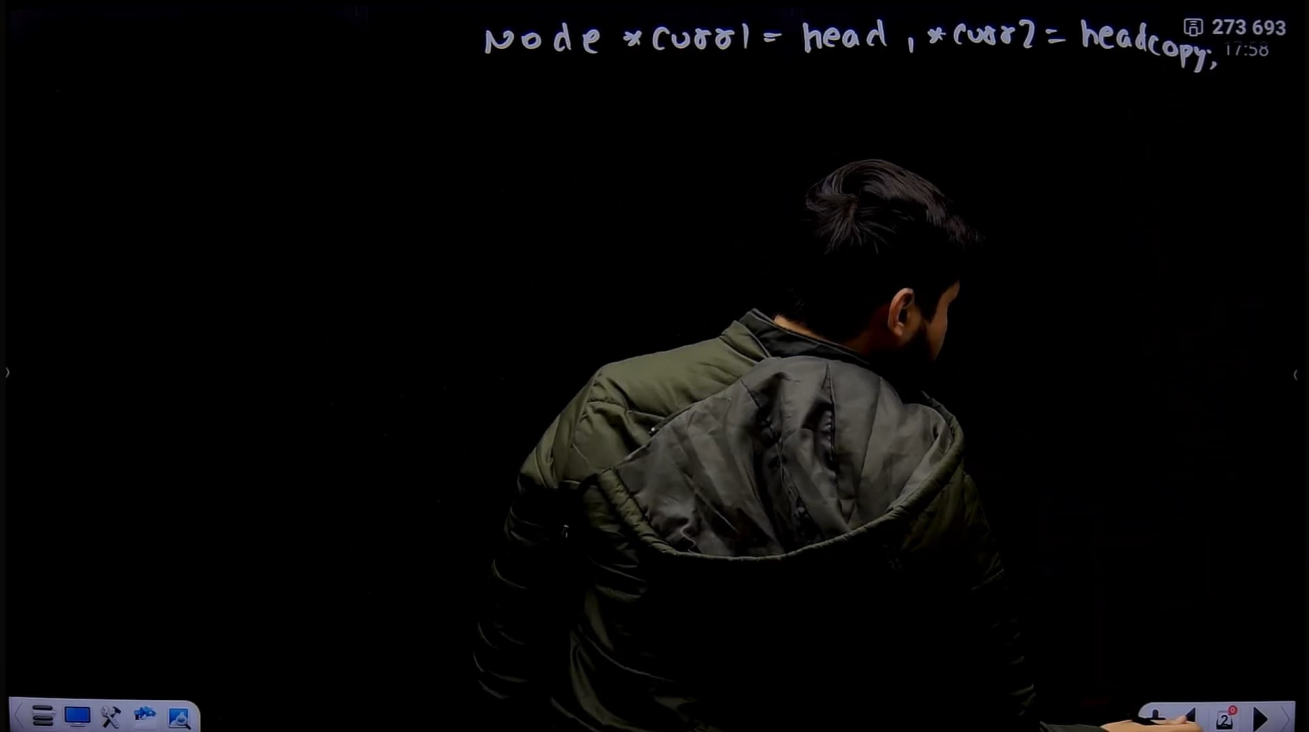



273 693
1
2
2
3
3
4
4
5
5
Copy

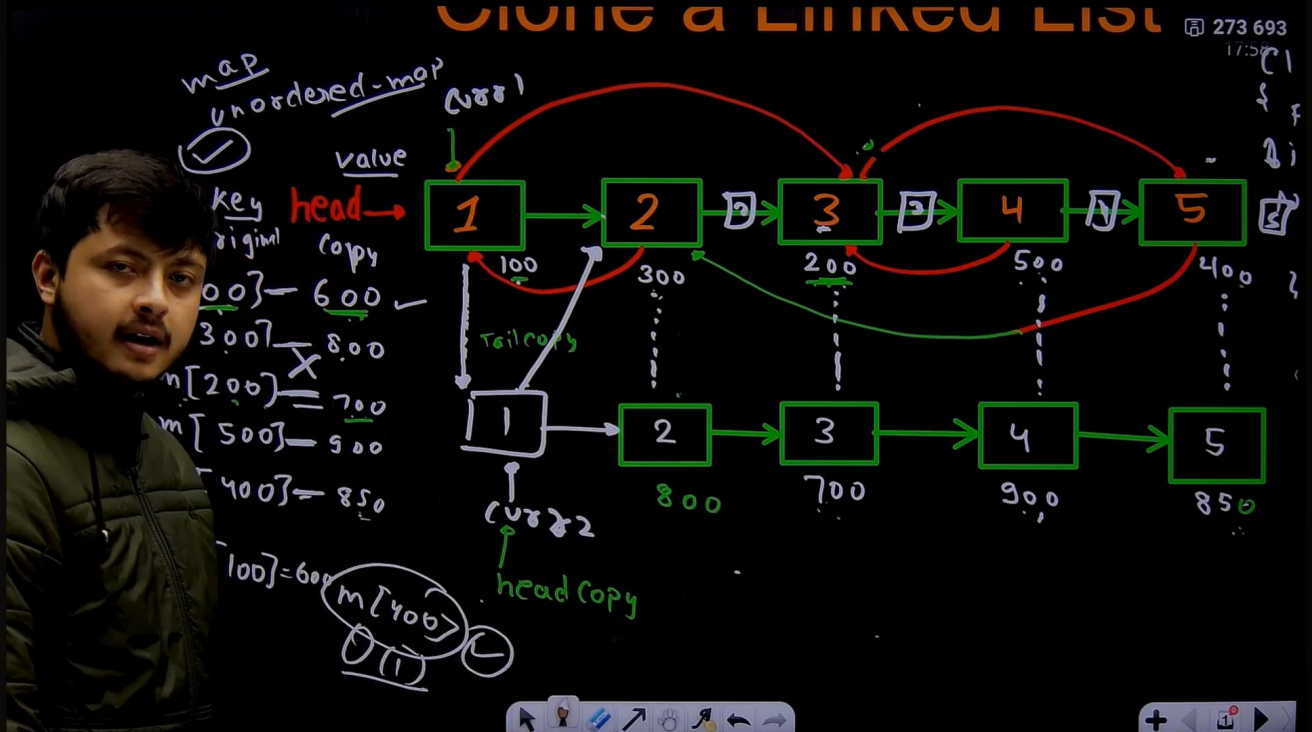



273 693
1
1
2
2
3
3
4
4
5
5
Curr
Curr1
Original
Copy
Curr2->arb = Curr1->arb->next;

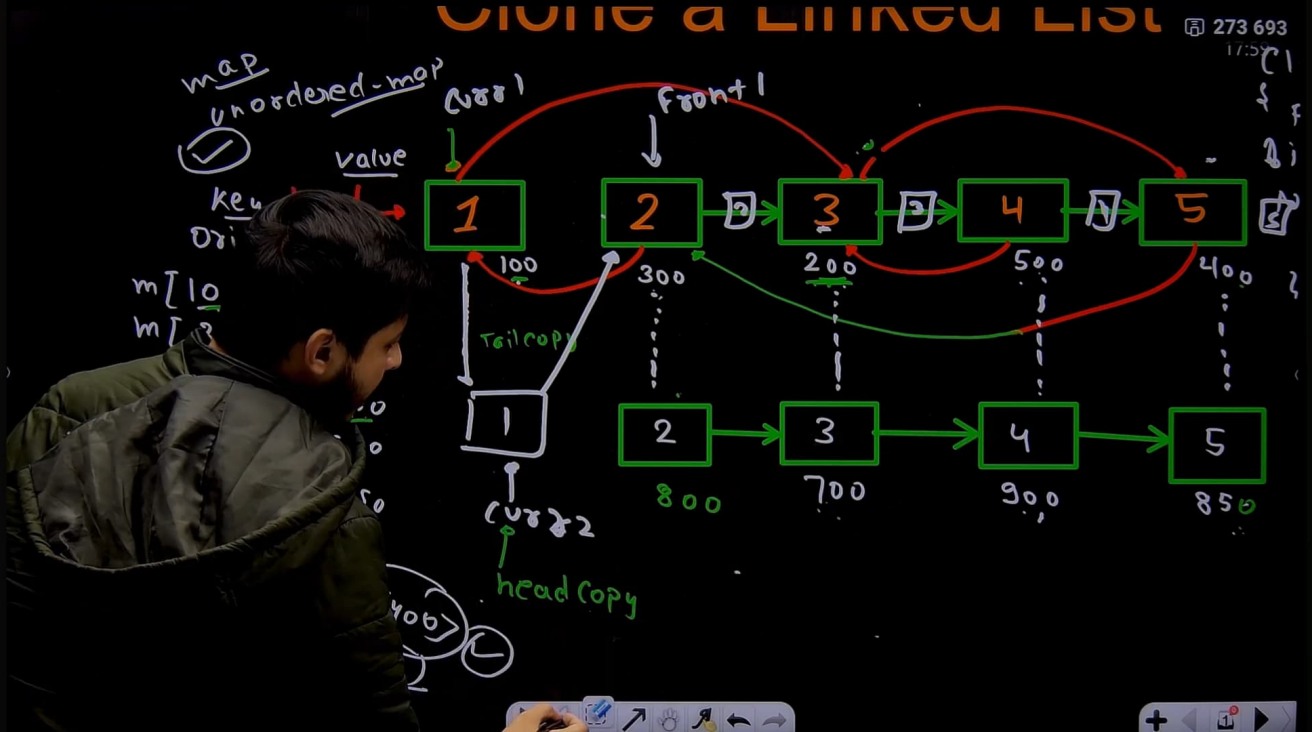

Unordered-map<Node*,Node*>m;
while(temp)
{
m[temp]= Tailcopy;
temp=temp->next;
tailcopy=tailcopy->next;
temp=head;
tailcopy=headcopy;
ile(temp)
->arb=
=tailc
next;
t;
ry:
Node * headcopy = new Node(0);
Node * tailcopy = headcopy;
Node * temp=head;
While(temp)
tailcopy->next = new Node(temp->data);
tailcopy=tailcopy->next;
temp=temp->next;
tailcopy=headcopy;
headcopy=headcopy->next;
delete tailcopy;
tailcopy=headcopy;
temp=head;
emp->arb];
O(n) Sc=O(n)
273 693
17:57


Node *curr1 = head , *curr2 = headcopy;
273 693
17:58

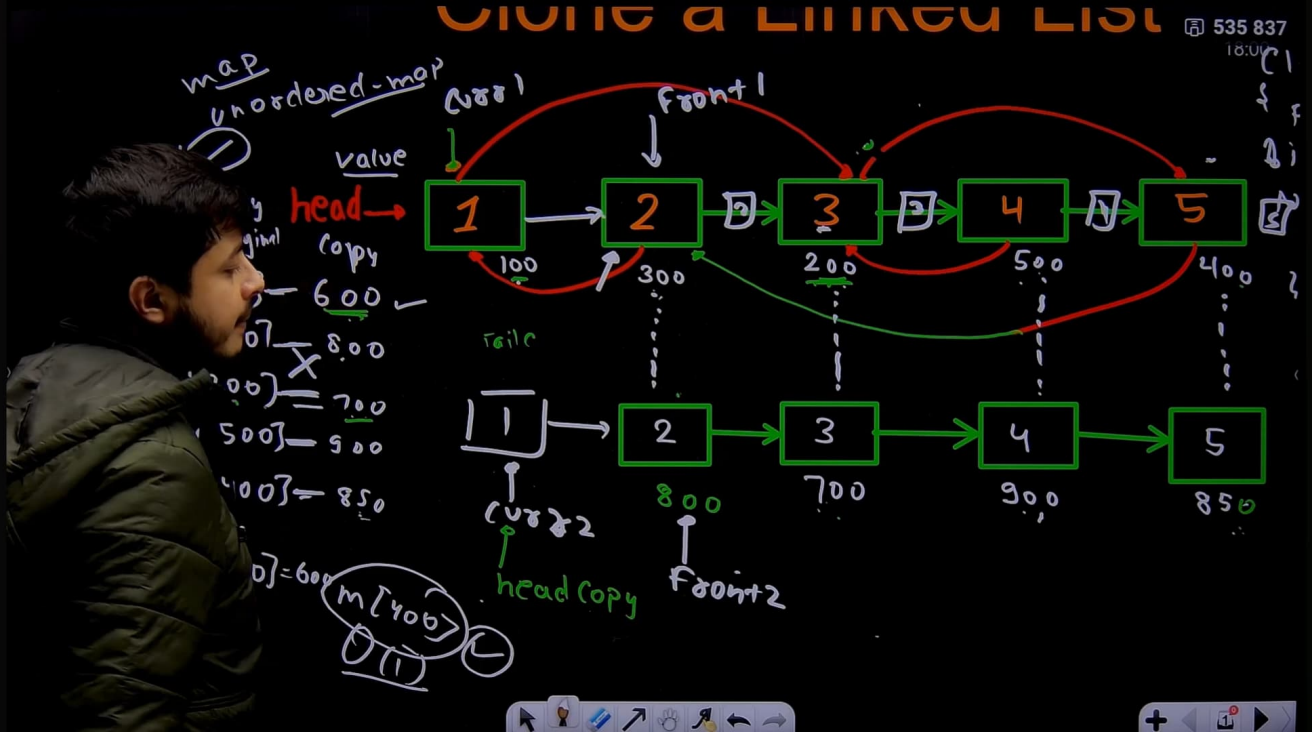
# Clone a Linked List



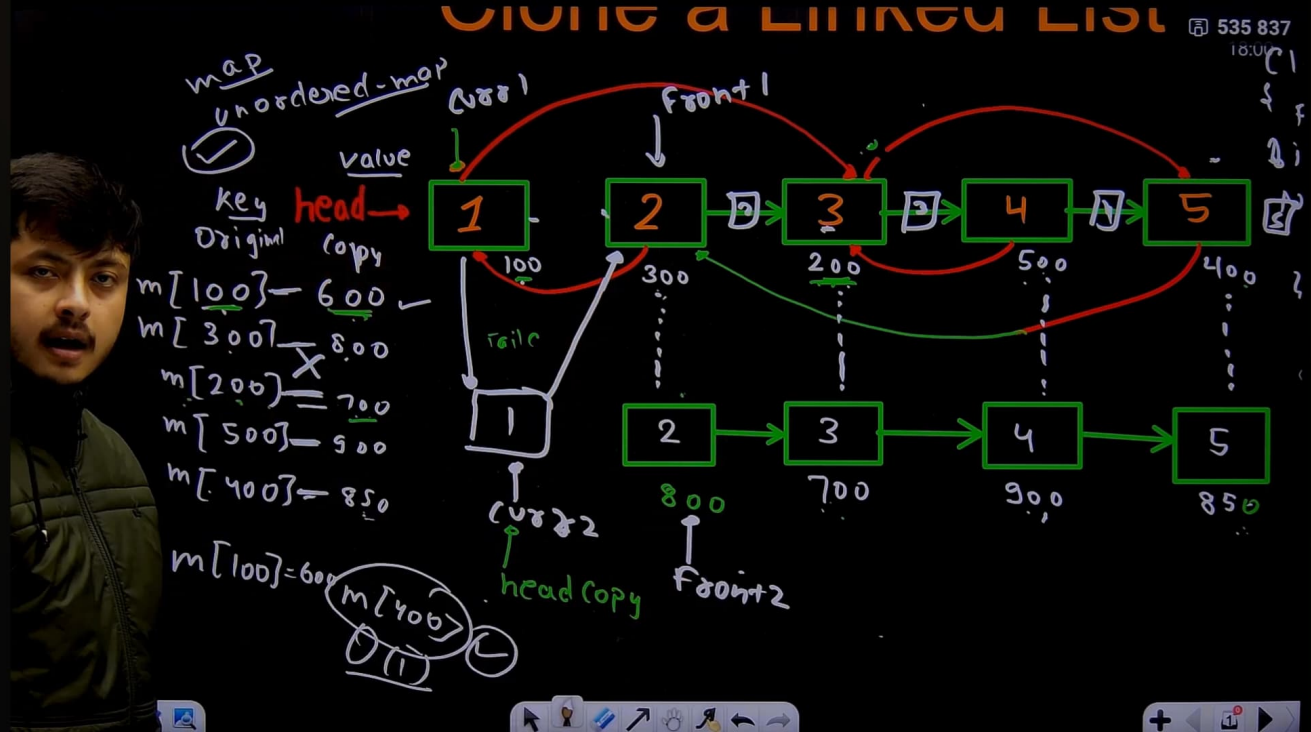
# Clone a Linked List

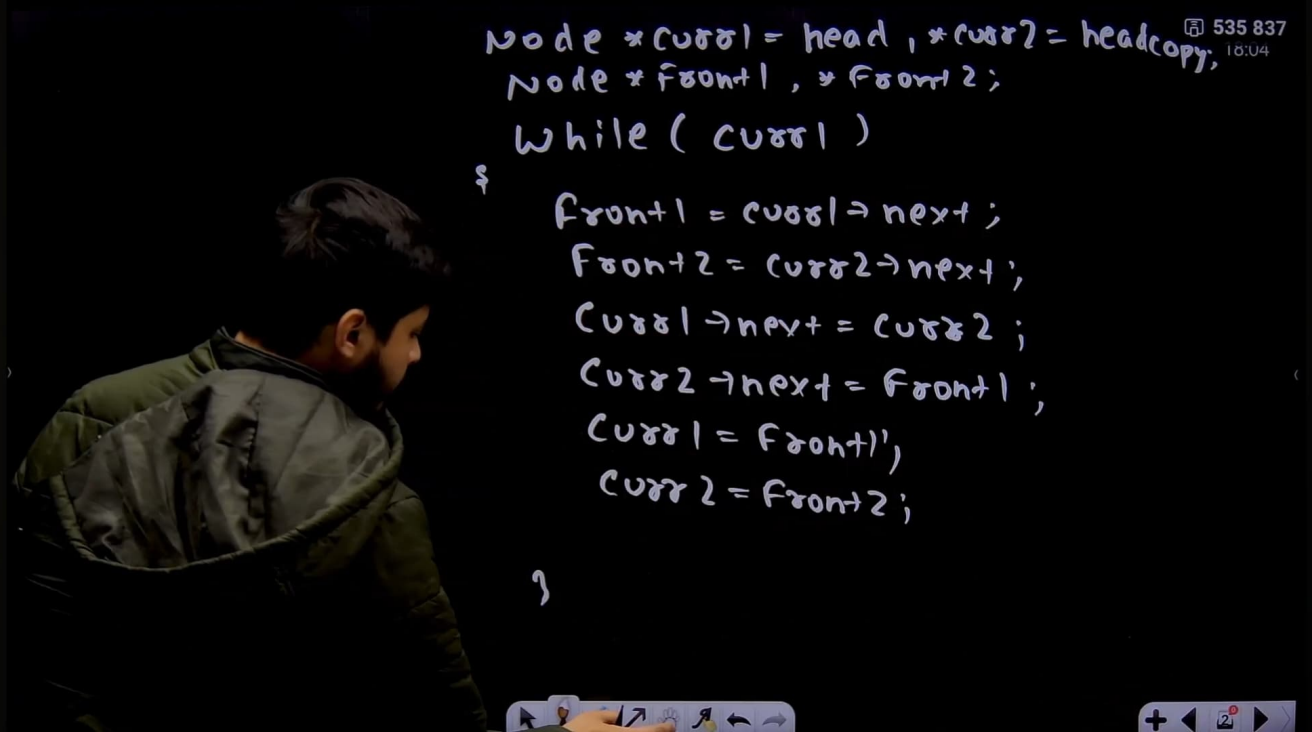


# Clone a Linked List

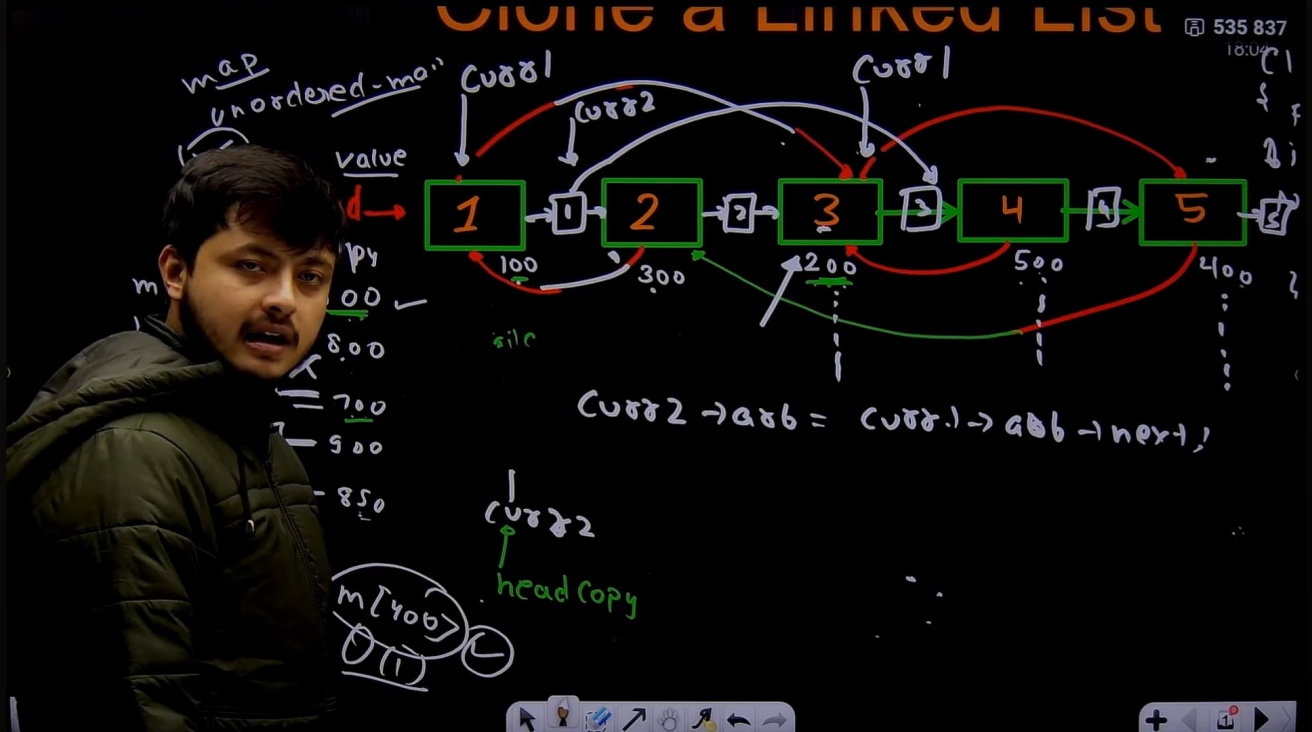


# Clone a Linked List



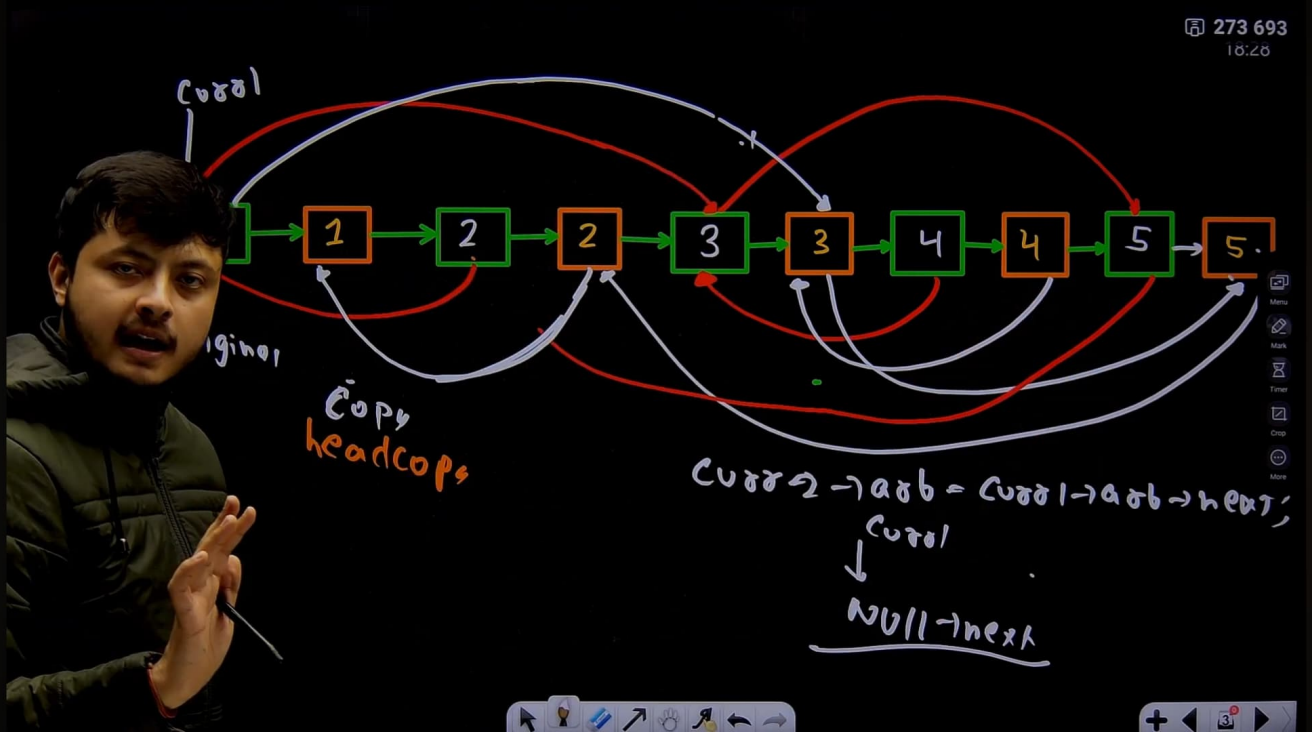

Node *curr1 = head , *curr2 = headcopy;
Node *front1 , *front2;
While ( curr1 )
{
front1 = curr1 → next ;
Front2 = curr2 → next ;
Curr1 → next = curr2 ;
Curr2 → next = Front1 ;
Curr1 = Front1 ;
Curr2 = Front2 ;
}
535 837
18:04

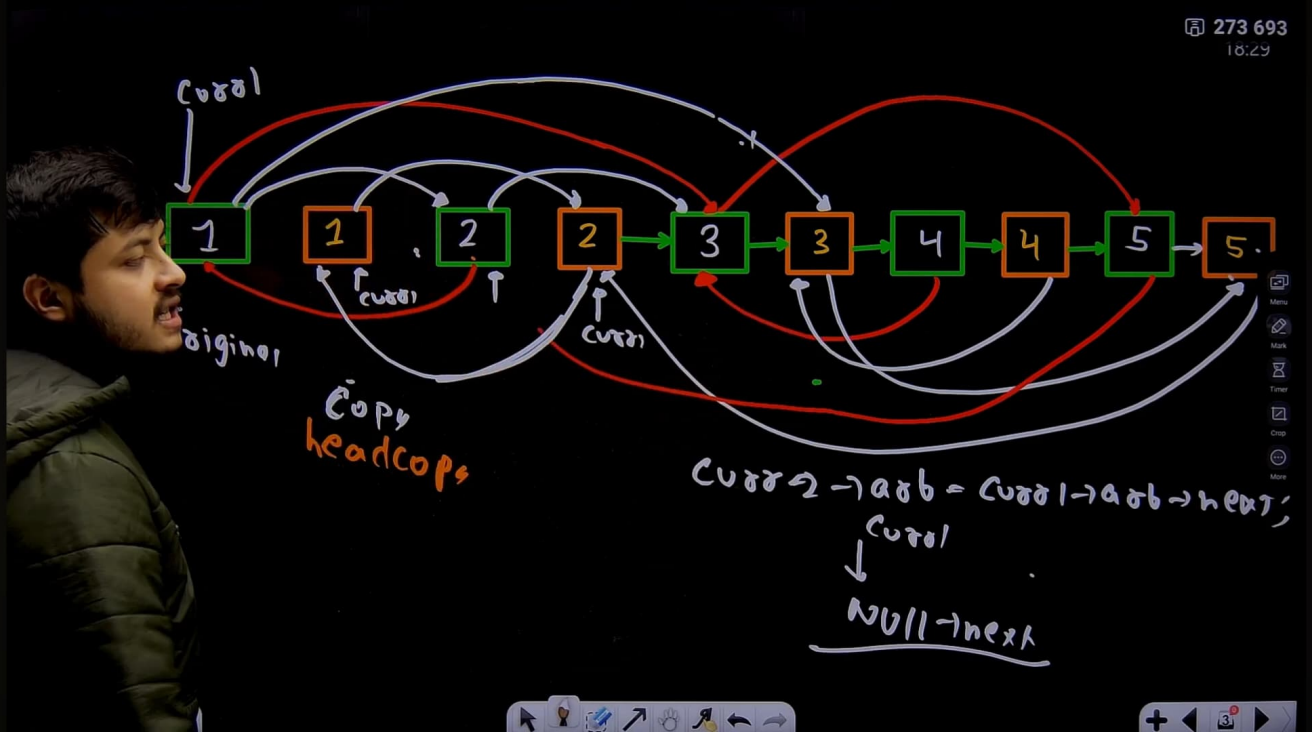
# Clone a Linked List

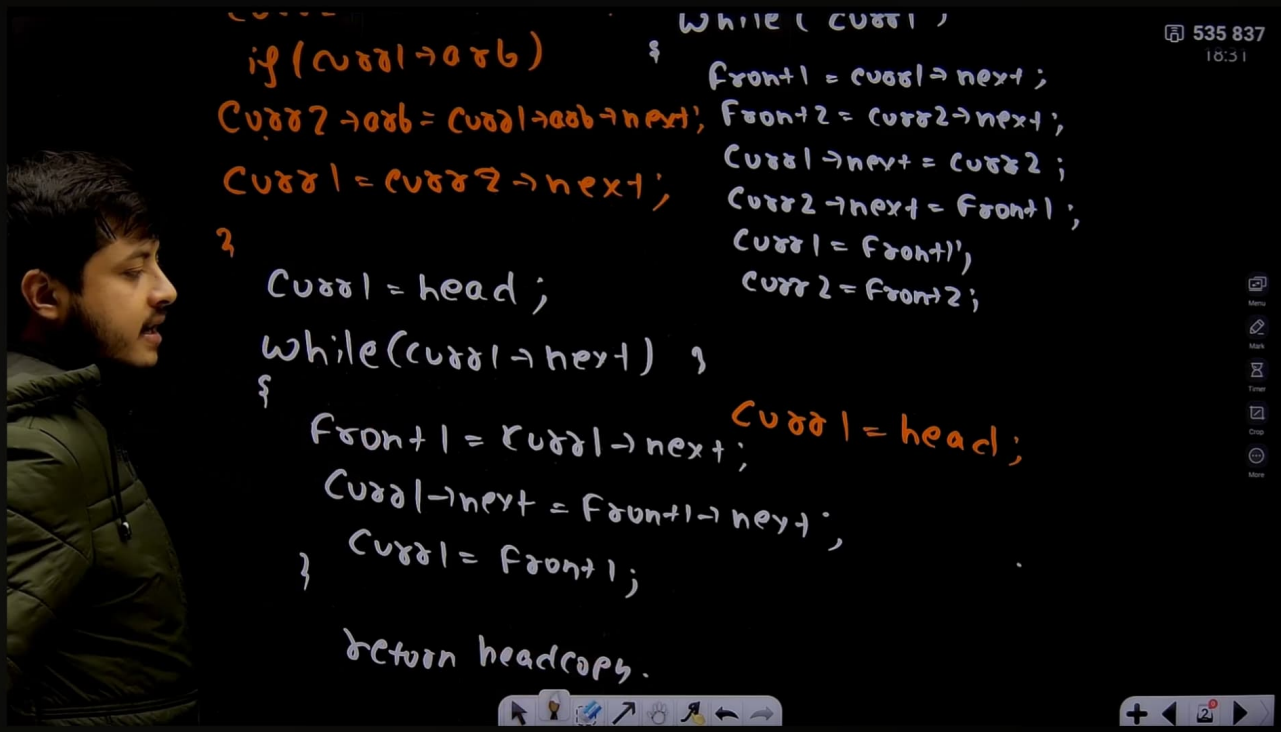


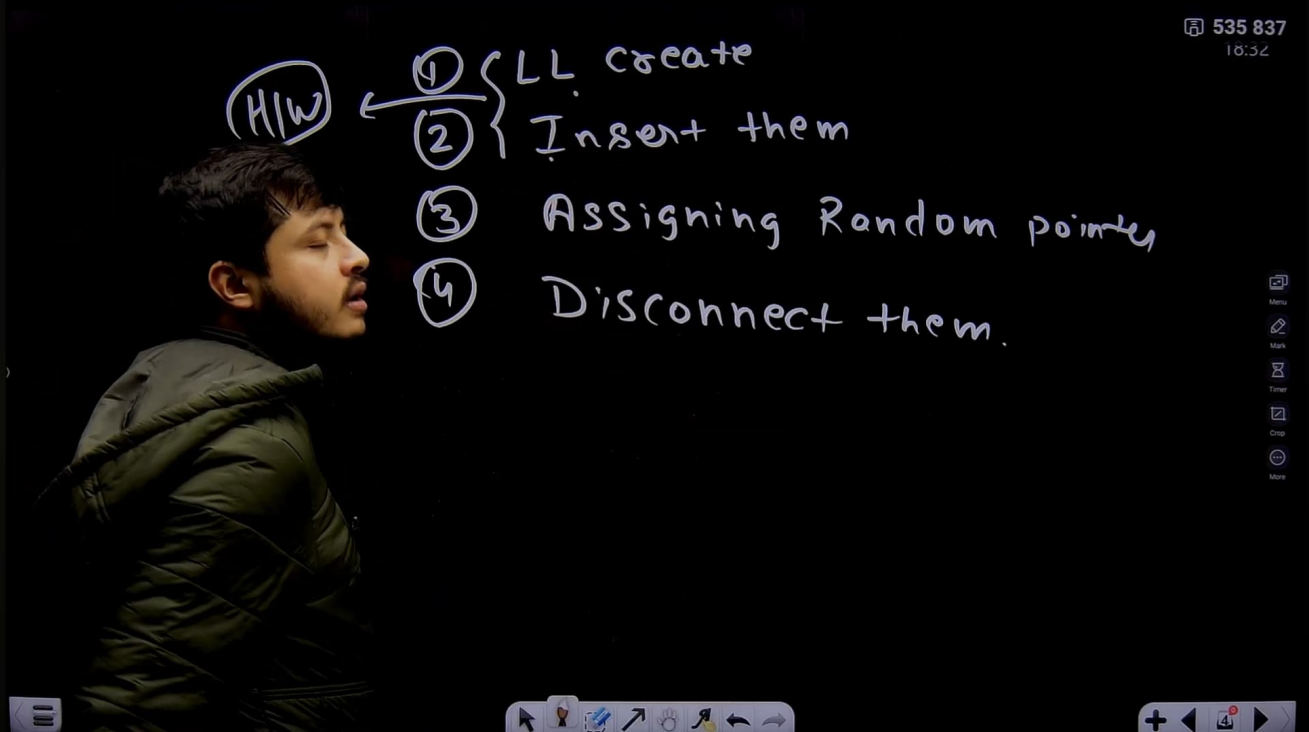
```
while(curr1)
{
  curr2 = curr1->next;
  if(curr1->arb)
  curr2->arb = curr1->arb->next;
  curr1 = curr2->next;
}
```

```
Node *curr1 = head, *curr2 = head;
Node *front1, *front2;
while ( curr1 )
{
  front1 = curr1->next;
  front2 = curr2->next;
  curr1->next = curr2;
  curr2->next = front1;
  curr1 = front1;
  curr2 = front2;
}
curr1 = head;
```


273 693
Curr1
1
2
2
3
3
4
4
5
5
Copy
headcopy
Curr2->arb = Curr1->arb->next;
Curr1
Null->next


273 693
18:29
curr1
1
1
2
2
3
3
4
4
5
5
curr1
curr2
original
Copy
headcopy
curr2->arb = curr1->arb->next;
curr1
Null->next


if (curr1->val)
curr2->val = curr1->val->next;
curr1 = curr2->next;
curr1 = head;
while(curr1->next)
front1 = curr1->next;
curr1->next = front1->next;
curr1 = front1;
return headcopy.
while (curr1)
front1 = curr1->next;
front2 = curr2->next;
curr1->next = curr2;
curr2->next = front1;
curr1 = front1;
curr2 = front2;
curr1 = head;


535 837
16:32
H/W
1
2
LL. create
Insert them
3
Assigning Random pointers
4
Disconnect them.
Menu
Mark
Timer
Crop
More

Problem | Editorial | Submissions | Comments

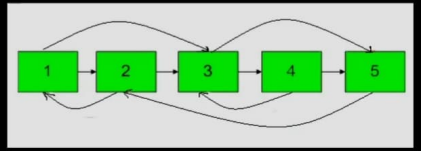


You are given a special linked list with **N** nodes where each node has a next pointer pointing to its next node. You are also given **M** random pointers, where you will be given **M** number of pairs denoting two nodes **a** and **b** i.e. **a->arb = b** (arb is pointer to random node).

Construct a copy of the given list. The copy should consist of exactly **N** new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes **X** and **Y** in the original list, where **X.arb --> Y**, then for the corresponding two nodes **x** and **y** in the copied list, **x.arb -> y**.

Return the head of the copied linked list.




C++ (g++ 5.4) | Start Timer

```
38              tailCopy->next = new Node(temp->data);
39              tailCopy = tailCopy->next;
40              temp=temp->next;
41          };
42
43          tailCopy = headCopy;
44          headCopy = headCopy->next;
45          delete tailCopy;
46
47          tailCopy = headCopy;
48          temp = head;
49
50          // Insert them in Orginal LL, merge donc
51
52
53
54
55
56      }
57
58  };
```

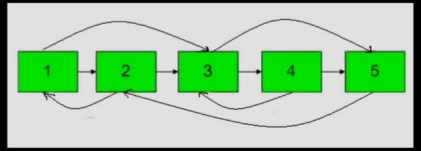
Problem | Editorial | Submissions | Comments

You are given a special linked list with **N** nodes where each node has a next pointer pointing to its next node. You are also given **M** random pointers, where you will be given **M** number of pairs denoting two nodes **a** and **b** **i.e. a->arb = b** (arb is pointer to random node).

Construct a copy of the given list. The copy should consist of exactly **N** new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes **X** and **Y** in the original list, where **X.arb --> Y**, then for the corresponding two nodes **x** and **y** in the copied list, **x.arb -> y.**

Return the head of the copied linked list.




**Note** :- The diagram isn't part of any example, it just depicts an example of how the linked list may look like.

C++ (g++ 5.4) | Start Timer

```
45      delete tailCopy;
46
47      tailCopy = headCopy;
48      temp = head;
49
50      // Insert them in Orginal LL, merge dono
51
52      Node *curr1 = head, *curr2 = headCopy;
53      Node *front1, *front2;
54
55
56      while(curr1)
57      {
58          front1 = curr1->next;
59          front2 = curr2->next;
60          curr1->next = curr2;
61          curr2->next = front1;
62          curr1=front1;
63          curr2 = front2;
64      }
65
```

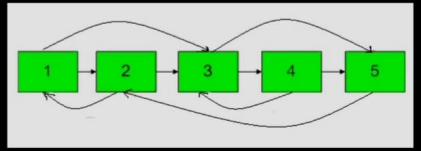
Problem | Editorial | Submissions | Comments

You are given a special linked list with **N** nodes where each node has a next pointer pointing to its next node. You are also given **M** random pointers, where you will be given **M** number of pairs denoting two nodes **a** and **b** **i.e. a->arb = b** (arb is pointer to random node).

Construct a copy of the given list. The copy should consist of exactly **N** new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes **X** and **Y** in the original list, where **X.arb --> Y**, then for the corresponding two nodes **x** and **y** in the copied list, **x.arb -> y.**

Return the head of the copied linked list.




**Note** :- The diagram isn't part of any example, it just depicts an example of how the linked list may look like.

C++ (g++ 5.4) | Start Timer

```
58          front1 = curr1->next;
59          front2 = curr2->next;
60          curr1->next = curr2;
61          curr2->next = front1;
62          curr1=front1;
63          curr2 = front2;
64      }
65
66
67      // Assign random pointer to cloned LL
68      curr1 = head;
69
70      while(curr1)
71      {
72          curr2 = curr1->next;
73          if(curr1->arb)
74          curr2->arb = curr1->arb->next;
75          curr1 = curr2->next;
76      }
77
78
```

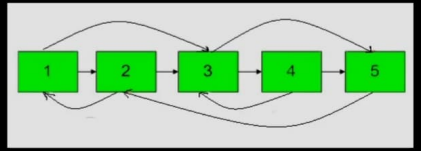
Problem | Editorial | Submissions | Comments

You are given a special linked list with **N** nodes where each node has a next pointer pointing to its next node. You are also given **M** random pointers, where you will be given **M** number of pairs denoting two nodes **a** and **b** i.e. **a->arb = b** (arb is pointer to random node).

Construct a copy of the given list. The copy should consist of exactly **N** new nodes, where each new node has its value set to the value of its corresponding original node. Both the next and random pointer of the new nodes should point to new nodes in the copied list such that the pointers in the original list and copied list represent the same list state. None of the pointers in the new list should point to nodes in the original list.

For example, if there are two nodes **X** and **Y** in the original list, where **X.arb --> Y**, then for the corresponding two nodes **x** and **y** in the copied list, **x.arb -> y.**

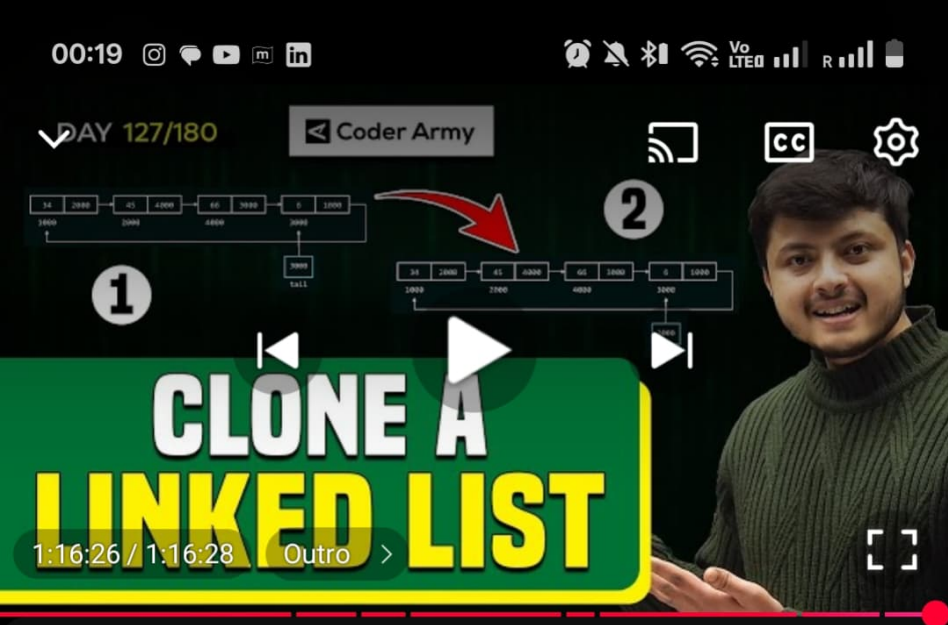
Return the head of the copied linked list.




**Note** :- The diagram isn't part of any example, it just depicts an example of how the linked list may look like.

C++ (g++ 5.4) Start Timer

```
71      {
72          curr2 = curr1->next;
73          if(curr1->arb)
74          curr2->arb = curr1->arb->next;
75          curr1 = curr2->next;
76      }
77
78
79      // Break the LL
80
81      curr1 = head;
82
83      while(curr1->next)
84      {
85          front1 = curr1->next;
86          curr1->next = front1->next;
87          curr1 = front1;
88      }
89
90      return headCopy;
91
```




## Replies

@safalpatel2839 • 4s ago

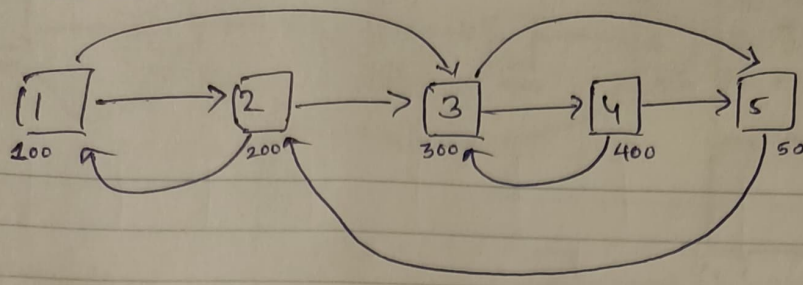
Thank you so much rohit 🙌,
The way you solve problem by starting from brute -force to go optimizing step-by-step,
shows how important it is to
1. understand the problem clearly,

2. break that problem down into multiple steps,

3. then deriving soln,
and
then for optimizing it
i). why exactly or what part of code is taking more time to execute
& then optimizing that particular block of code,
& most importantly,
understanding why/need of particular subproblem & then addressing it, for ex in optimized approach,
we wanted to store the info that
addr 100 of list1 corresponds to addr 600 of list2,
how to store that without taking help of an extra data structure, so we made changes into arrangments so to get help in remembering (this info) without actually utilizing extra space.

Reply...

# Clone a LL



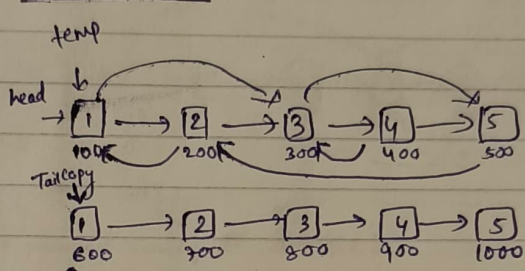

```
Class Node {
public:
int data
Node *next
*arb
};
```

## Approach 1: Brute-force

1. Create a LL without random pointer.
2. Assign random pointer from start.

T.C: O(n).
S.C: O(n).

## Approach 2:




Key — Value

| Original | Copy |
|---|---|
| 100 | 600 |
| 200 | 700 |
| 300 | 800 |
| 400 | 900 |
| 500 | 1000 |

Somehow, we have this address mapping which helps us get the corresponding addr in the cloned LL in O(1) time.

map — Before this: (P2)

```
unordered_map <Node*, Node*> m;
while(temp){
m[temp] = tailCopy;
temp = temp->next;
tailCopy = tailCopy->next;
}
temp = head;
```

### P1:

```
Node* headCopy = new Node(0);
Node* tailCopy = headCopy;
Node* temp = head;
while(temp){
  tailCopy->next = new Node(temp->data);
  tailCopy = tailCopy->next
  temp = temp->next;
}
tailCopy = headCopy;
headCopy = headCopy->next;
delete tailCopy;
```

Approach 1:
T.C: $O(n^2)$ S.C: O(1).

### P2:

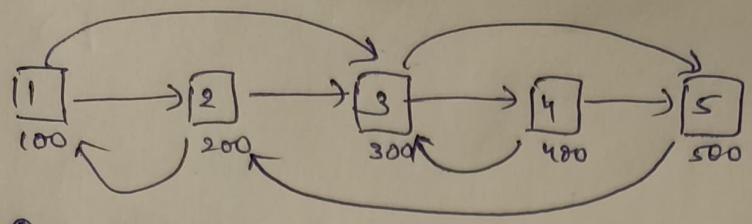
```
tailCopy = headCopy, temp = head;
while(temp){
  tailCopy->arb = find(head, headCopy, temp->arb);
  tailCopy = tailCopy->next;
  temp = temp->next;
}
return headCopy;
```

$O(n^2)$

### Find function

```
Node* find(Node* curr1, Node* curr2, Node* x){
  if(x==NULL)
  return NULL;
  while(curr1 != x){
    curr1 = curr1->next;
    curr2 = curr2->next;
  }
  return curr2;
}
```

n




4 steps:

P1: Create LL without *arb.

P2: Insert clonedLL nodes into original.

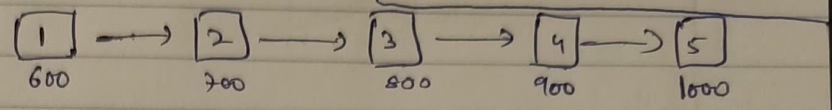
P3: Assigning *arb to the nodes in the cloned LL.

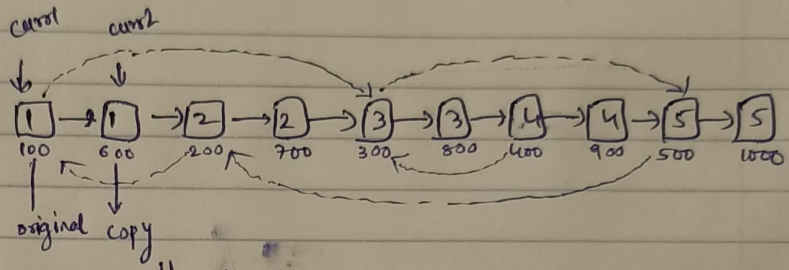
P4: Disconnect clone list from the original list.

T.C: O(n), S.C: O(1).

## Approach 3: Optimized Approach:

How to get corresponding addr of a node in a cloned LL (100 - 600) without actually having to store it.



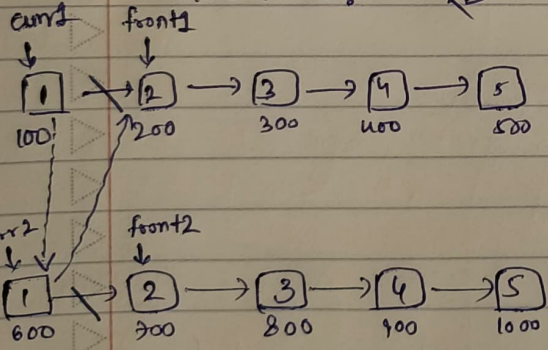

Can we make some changes in the Arrangements itself?

What if we place **original node & clone node** together in a LL

How to do?

* LOGIC:

```
Clone Node -> arb = original Node -> arb -> next;
curr2 -> arb = curr1 -> arb -> next.
```




P3: Assigning arb pointer to the cloned LL.

```
curr1 = head, curr2 = headCopy
while (curr1) {
    curr2 = curr1 -> next;
    if (curr1 -> arb)
    curr2 -> arb = curr1 -> arb -> next;
    curr1 = curr2 -> next;
}
```

P2: Logic: Inserting them into an original LL.

```
Node * curr1 = head, * curr2 = headCopy;
Node * front1, * front2;
while (curr1) {
    front1 = curr1 -> next;
    front2 = curr2 -> next;
    curr1 -> next = curr2;
    curr2 -> next = front1;
    curr1 = front1;
    curr2 = front2;
}
```

P4: Disconnecting original from clone LL.
(whether original node / clone node, disjoining happens in same manner).

```
curr1 = head;
while (curr1 -> next) {
    front1 = curr1 -> next;
    curr1 -> next = front1 -> next;
    curr1 = front1;
}
return headCopy.
```